

Spécification du protocole — Infrastructure de collecte d'ambiance

IFT3225 — Phase 1 · Rapport d'équipe

Lieu instrumenté : Cafétéria Roger-Gaudry, Université de Montréal

Ce rapport décrit les règles de communication avec notre système : les ressources et leurs endpoints, les conventions de format, le mécanisme d'authentification, la chaîne de collecte, la logique d'agrégation des vues sémantiques, ainsi que les limites assumées et les évolutions envisagées. Le protocole est une API REST versionnée en JSON. Il sépare les ressources persistées (appareils, lieux, mesures, observations) des vues sémantiques calculées (ambiance courante, créneaux calmes, comparaison de lieux, historique agrégé).

1. Lieu et plan de collecte

Lieu choisi : **Cafétéria Roger-Gaudry** (Université de Montréal). Objectif : évaluer l'ambiance du lieu à partir de mesures sonores et d'observations humaines, afin de déterminer s'il est calme, modéré, animé ou bruyant selon les moments de la journée.

La collecte est mixte. Le niveau sonore est mesuré automatiquement au téléphone (Phyphox) ; les données environnementales (densité, proximité de la source de bruit, vibe ressentie) sont saisies par un observateur. Au moins trois sessions de 20 minutes sont réalisées à des moments différents de la journée et de la semaine.

Mode	Moyen	Description
Automatique	Téléphone avec Phyphox	Mesures régulières du niveau sonore en dB pendant une session.
Manuelle	Formulaire de saisie	Observation de la densité, de la proximité, de l'ambiance ressentie et notes.
Fallback obligatoire	Saisie manuelle	Utilisée si la collecte automatique échoue, si le capteur est indisponible ou si le réseau est instable.

2. Ressources et endpoints

Les chemins commencent par /v1 afin de permettre l'évolution future du protocole sans casser les clients existants. Le découpage en ressources distingue les entités persistées des vues dérivées.

2.1 Justification du découpage en ressources

Ressource	Pourquoi ce regroupement	Alternative envisagée et décision
devices	Un appareil représente une source de collecte identifiable, porteuse d'un identifiant et d'une clé API.	Mettre l'appareil uniquement dans measurements éviterait des doublons mais empêcherait de révoquer une clé ou de suivre lastSeenAt. Écarté.

Ressource	Pourquoi ce regroupement	Alternative envisagée et décision
locations	Un lieu est une entité stable que les clients consultent et filtrent.	Écrire le lieu comme simple chaîne dans chaque mesure crée fautes et doublons et rend les comparaisons difficiles. Écarté.
measurements	Donnée numérique brute produite par capteur : type, valeur, unité, timestamp.	Fusionner avec observations a été écarté : une mesure est objective et numérique, une observation est qualitative.
observations	Donnée humaine : densité, proximité, vibe, notes.	La stocker comme mesure textuelle compliquerait la validation et les statistiques. Écarté.
ambiance	Vue calculée sur les données existantes ; répond directement aux questions du client.	N'est pas une ressource persistée : score, ambianceLabel et quietSlots sont dérivés à la lecture.

2.2 Gestion des appareils

Méthode	Endpoint	Corps / paramètres	Réponse	Codes
POST	/v1/devices	{ name, locationSlug }	201 + { id, name, locationSlug, apiKey }	201, 400, 404, 409
GET	/v1/devices	locationSlug?, page?, perPage?, sort?	200 + tableau { id, name, locationSlug, lastSeenAt }	200
DELETE	/v1/devices/{id}	Aucun corps. Clé admin requise.	204 — clé révoquée, appareil supprimé	204, 401, 403, 404

2.3 Gestion des lieux

Méthode	Endpoint	Corps / paramètres	Réponse	Codes
GET	/v1/locations	city?, type?, page?, perPage?	200 + tableau { slug, displayName, city, type }	200
POST	/v1/locations	{ slug, displayName, city, type }	201 + lieu créé	201, 400, 401, 403, 409
PUT	/v1/locations/{slug}	{ displayName?, city?, type? }	200 + lieu mis à jour	200, 400, 401, 403, 404

2.4 Collecte des données

Méthode	Endpoint	Corps	Réponse	Codes
POST	/v1/measurements	{ type, value, unit, locationSlug, timestamp, deviceId? }	201 + mesure créée (+ receivedAt)	201, 400, 401, 403, 404, 422
POST	/v1/observations	{ locationSlug, density, proximity, vibe, notes?, timestamp }	201 + observation créée	201, 400, 401, 403, 404, 422
POST	/v1/measurements/batch	[{ type, value, unit, locationSlug, timestamp }, ...]	207 + { accepted[], rejected[] }	207, 400, 401, 403

2.5 Consultation brute

Méthode	Endpoint	Filtres	Codes
GET	/v1/measurements	locationSlug?, type?, from?, to?, last?, page?, perPage?, sort?	200, 400
GET	/v1/observations	locationSlug?, vibe?, density?, from?, to?, last?, page?, perPage?, sort?	200, 400

2.6 Endpoints sémantiques (vues dérivées)

Ces endpoints dépassent la simple collecte : ils transforment les données brutes en réponses directement utiles. Ils ne stockent rien et sont publics en lecture.

Méthode	Endpoint	Paramètres	Ce qu'il retourne
GET	/v1/ambiance/{slug}/now	window? = 15m 30m 1h (défaut 30m)	Portrait actuel : bruit moyen, occupation, vibe et proximité dominantes, ambianceLabel.
GET	/v1/ambiance/{slug}/quiet-hours	days? = 7 14 30 ; threshold? (dB) ; dayOfWeek? = 0-6	Créneaux typiquement calmes (par tranches de 30 min) avec avgNoise.
GET	/v1/ambiance/compare	locations (slugs séparés par virgule) ; window?	Comparaison de plusieurs lieux + quietest + busiest.
GET	/v1/ambiance/{slug}/history	last? ou from/to ; bucket? = 5m 15m 30m 1h	Série temporelle agrégée par tranche.

Exemple de réponse — ambiance courante

```
{
  "status": "success",
  "data": {
    "location": "cafeteria-roger-gaudry",
    "window": "30m",
    "sampleSize": { "measurements": 24, "observations": 1 },
    "score": { "noise": 72, "occupancy": 70, "vibe": "Sociable",
              "proximity": "Fréquenté" },
    "ambianceLabel": "animé"
  },
  "meta": { "generatedAt": "2026-06-01T18:32:10Z" }
}
```

Stratégie de gestion des lieux

Création explicite. Un lieu doit exister (créé via POST /v1/locations) avant qu'une mesure ou un appareil ne le référence. Une mesure dont le locationSlug est inconnu reçoit 404 LOCATION_NOT_FOUND. Ce choix évite la prolifération de lieux fantômes créés par une faute de frappe, au prix d'une étape d'enregistrement préalable.

3. Conventions du protocole

Convention	Choix retenu	Intention
Versionnement	Tous les chemins commencent par /v1.	Permet de créer /v2 plus tard sans casser les clients existants.
Nommage	Noms anglais, pluriels pour les collections. Slugs en kebab-case (cafeteria-roger-gaudry).	Rend les URLs prévisibles, stables et faciles à filtrer.
Format des dates	ISO 8601 en UTC avec suffixe Z. Une heure locale doit inclure son offset (...-04:00).	Évite les ambiguïtés de fuseau horaire et d'heure d'été.
timestamp vs receivedAt	Le client fournit timestamp (moment réel de la mesure) ; le serveur ajoute receivedAt (réception).	Permet de distinguer une mesure faite à 14 h d'une mesure envoyée plus tard en batch.
Réponses	JSON uniquement, Content-Type: application/json.	Format uniforme pour tous les clients.
Enveloppe de succès	{ status, data, meta }	Le client sait toujours où trouver les données et les métadonnées.
Enveloppe d'erreur	{ status: "error", error: { code, message, details? }, meta }	Message humain + code machine traitable automatiquement.
Pagination	page, perPage, total, totalPages dans meta pour les collections.	Évite les réponses trop lourdes ; rend l'interface plus rapide.

3.1 Structure des erreurs

```
{
  "status": "error",
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Le champ value est obligatoire.",
    "details": [ { "field": "value", "issue": "missing" } ]
  },
  "meta": { "generatedAt": "2026-06-01T18:11:24Z" }
}
```

Statut	Code machine	Cas
400	VALIDATION_ERROR	Champ manquant, mauvais format, ou combinaison invalide de paramètres.
401	MISSING_AUTH	En-tête d'authentification absent.
403	FORBIDDEN	Clé invalide ou accès refusé.
404	LOCATION_NOT_FOUND / NOT_FOUND	Lieu ou ressource inexistant.
409	DEVICE_EXISTS / LOCATION_EXISTS	Doublon de nom, de slug ou d'appareil.
422	INVALID_VALUE	Valeur hors plage ou incompatible avec son type.
429	RATE_LIMIT	Trop de requêtes (limite par minute atteinte).
500	INTERNAL_ERROR	Erreur serveur non prévue.

3.2 Types et valeurs validées

Champ	Valeurs acceptées	Rôle
measurement.type	noise_level	Seul type supporté en Phase 1 (niveau sonore).
measurement.unit	dB	Unité du décibel.
measurement.value	0 à 140	Plage valide du niveau sonore.
observation.density	Vide Modéré Fréquenté Bondé	Densité perçue du lieu.
observation.vibe	Calme Concentré Sociable Bruyante Festive Tendue	Ambiance humaine ressentie.
observation.proximity	Isolé Espacé Fréquenté Serré	Proximité de la source de bruit humaine.

La validation distingue deux niveaux : un champ manquant ou mal typé renvoie 400 `VALIDATION_ERROR` ; une valeur bien typée mais hors domaine (ex. value = 200, ou vibe inconnue) renvoie 422 `INVALID_VALUE`.

3.3 Filtrage, consultation et pagination

Les endpoints qui retournent des collections partagent les mêmes paramètres de filtrage, afin que le client réutilise la même logique sur plusieurs écrans.

Paramètre	Type	Exemple	Description
locationSlug	string	?locationSlug=cafeteria-roger-gaudry	Filtrer par lieu exact.
from / to	ISO 8601	?from=2026-06-01T00:00:00Z	Bornes de période.
last	durée	?last=3h	Raccourci temporel. Exclusif avec from/to.
page / perPage	entier	?page=2&perPage=25	Pagination. perPage max : 200.
sort	string	?sort=timestamp:desc	Tri au format champ:asc champ:desc.

Règle importante : combiner from/to et last dans la même requête renvoie 400 `VALIDATION_ERROR`, car cela définirait deux fenêtres temporelles contradictoires. La réponse paginée place les métadonnées dans meta : page, perPage, total, totalPages.

4. Authentification

Les endpoints d'écriture sont protégés par une clé API transmise dans l'en-tête HTTP `x-api-key`. Les endpoints de lecture (GET) restent publics. Deux familles de clés coexistent.

<code>x-api-key: dev_3f9a1c...</code>	(clé d'appareil – endpoints de collecte)
<code>x-api-key: <ADMIN_API_KEY></code>	(clé d'administration – endpoints de gestion)

Note de conformité : une version antérieure de ce rapport évoquait `Authorization: Bearer`. La consigne de la Tâche 5 impose `x-api-key` ; *c'est ce que l'implémentation et la présente spécification retiennent.*

4.1 Portée des clés

Type de clé	Endpoints protégés	Vérification
Clé d'appareil (apiKey)	POST /measurements, POST /observations, POST /measurements/batch	La clé doit correspondre à un device enregistré ; lastSeenAt est mis à jour.
Clé d'administration (ADMIN_API_KEY)	DELETE /devices/{id}, POST /locations, PUT /locations/{slug}	La clé doit égaler le secret ADMIN_API_KEY du serveur.
Aucune (public)	Tous les GET, y compris les vues /ambiance	Lecture ouverte.

4.2 Flux complet

- **Enregistrement** — un administrateur crée un appareil via POST /v1/devices et reçoit, une seule fois, sa apiKey.
- **Utilisation** — le bridge (ou tout client capteur) envoie chaque mesure avec l'en-tête x-api-key: <apiKey>.
- **Révocation** — DELETE /v1/devices/{id} (clé admin) supprime l'appareil ; sa clé devient aussitôt invalide.

Comportement en cas de clé absente ou invalide

Situation	Statut	Code machine
En-tête x-api-key absent	401	MISSING_AUTH
Clé présente mais ne correspond à aucun device / au secret admin	403	FORBIDDEN
Clé valide	200 / 201 / 207	— (requête autorisée)

La faille volontaire de POST /devices (non protégé en Phase 1) est analysée en section 7.1.

5. Collecte

5.1 Du capteur au serveur : le bridge

La donnée sonore va du téléphone au serveur via un **bridge** (bridge/bridge.js). Phyphox, avec « Allow remote access » activé, expose ses buffers sur http://<ip-tel>:8080/get?<buffer>. Le bridge interroge ce point d'accès à intervalle régulier, lit la dernière valeur du buffer sonore, puis POST une mesure vers /v1/measurements avec l'en-tête x-api-key.

Pourquoi un bridge ? Il découple la collecte du serveur : le téléphone n'a pas à connaître MongoDB ni la logique métier ; il expose seulement ses buffers. Le bridge joue le rôle de client capteur, applique l'authentification et le format du protocole. Une alternative — faire pousser directement le téléphone vers MongoDB — a été écartée : elle exposerait les identifiants de base et contournerait la validation du serveur.

Paramètre	Valeur	Rôle
POLL_INTERVAL_MS	5000 (défaut)	Fréquence d'interrogation du capteur : une mesure toutes les 5 s.
PHYPHOX_URL	http://<ip-tel>:8080	Adresse de l'API locale Phyphox sur le réseau Wi-Fi.
PHYPHOX_BUFFER	dB	Buffer sonore lu sur le capteur.
LOCATION_SLUG	cafeteria-roger-gaudry	Lieu rattaché aux mesures envoyées.

5.2 Envoi groupé (batch)

Lorsque la collecte accumule des mesures hors ligne (réseau instable), elle peut les envoyer en lot via `POST /v1/measurements/batch`, dont le corps est un tableau de mesures. La réponse 207 rapporte un succès partiel : `{ accepted[], rejected[] }`. Chaque élément rejeté indique son index et son code d'erreur, sans bloquer les mesures valides du même lot. C'est ici que la distinction `timestamp / receivedAt` prend son sens : le moment réel de la mesure est préservé même si l'envoi est différé.

5.3 Données environnementales et fallback manuel

Les données environnementales (densité, proximité, vibe, notes) ne proviennent pas d'un capteur : elles sont saisies par un observateur via `POST /v1/observations`. Cette saisie manuelle constitue aussi le **fallback obligatoire** : si Phyphox est indisponible, si le téléphone manque ou si le Wi-Fi est instable, l'équipe consigne malgré tout l'ambiance du lieu, garantissant qu'aucune session ne reste vide. Une session type dure 20 minutes : le bridge alimente le bruit en continu pendant qu'un membre enregistre une ou deux observations.

6. Agrégation des vues sémantiques

Les endpoints `/ambiance` ne stockent rien : ils agrègent à la lecture les mesures et observations d'une fenêtre temporelle. Voici la logique appliquée.

6.1 Étiquette d'ambiance (ambianceLabel)

L'étiquette est dérivée du niveau sonore moyen de la fenêtre, selon des seuils en décibels :

Bruit moyen	Étiquette
< 50 dB	calme
50 à < 65 dB	modéré
65 à < 75 dB	animé
≥ 75 dB	bruyant

En l'absence de mesure dans la fenêtre, l'étiquette vaut « inconnu ».

6.2 Score d'occupation et dominantes

L'occupation est calculée à partir des observations : chaque densité est convertie en score (Vide = 10, Modéré = 40, Fréquenté = 70, Bondé = 95), puis moyennée. La `vibe` et la `proximity` dominantes sont obtenues par mode (valeur la plus fréquente). Le `portrait` renvoie aussi `sampleSize` (nombre de mesures et d'observations agrégées) pour signaler la fiabilité de la fenêtre.

6.3 Créneaux calmes (quiet-hours)

Sur une période de `days` jours (7, 14 ou 30), les mesures sont regroupées par couple (jour de la semaine, tranche de 30 minutes). La moyenne de bruit de chaque tranche est calculée, puis on ne conserve que les tranches sous le `threshold` (défaut 55 dB), triées du plus calme au plus bruyant. Le paramètre `dayOfWeek` (0–6) restreint l'analyse à un jour précis.

6.4 Historique (history) et comparaison (compare)

`history` produit une série temporelle : les mesures de la fenêtre (`last` ou `from/to`, défaut 6 h) sont réparties en tranches de taille `bucket` (5m, 15m, 30m, 1h) et chaque tranche renvoie sa moyenne de bruit et son nombre d'échantillons. `compare` applique la même agrégation `now` à plusieurs lieux et désigne le plus calme (`quietest`) et le plus animé (`busiest`) par comparaison des bruits moyens.

7. Limites et évolution

7.1 Faille volontaire : POST /devices non protégé

En Phase 1, `POST /v1/devices` n'exige aucune authentification : n'importe qui peut créer un appareil et obtenir une `apiKey` valide, donc pousser de fausses mesures, fausser les vues sémantiques, et — par volume — déclencher le `rate-limit` pour les autres clients.

Solution proposée : exiger la clé d'administration sur `POST /devices`, comme pour les autres endpoints de gestion — le middleware `adminAuth` existe déjà et il suffit de l'ajouter à la route. Compléments envisageables : enrôlement par jeton d'invitation à usage unique, ou validation d'un compte propriétaire avant émission de la clé.

7.2 Gestion des secrets

Les secrets (URI MongoDB, clé admin) vivent dans `.env`, non versionné (présent dans `.gitignore`), avec un `.env.example` documenté. Limite identifiée : un secret accidentellement committé reste présent dans l'historique git même après retrait — il doit alors être révoqué et régénéré côté fournisseur (rotation du mot de passe Atlas), pas seulement supprimé du dernier commit.

7.3 Autres compromis assumés

- **Un seul type de mesure.** Seul `noise_level` est supporté. Le schéma (type/unit/value) est prêt à accueillir lumière, température ou CO₂ sans rupture.

- **Confiance au timestamp client.** Le serveur accepte le timestamp fourni ; un capteur mal réglé pourrait dater une mesure dans le futur. `receivedAt` permet de détecter les écarts, mais aucune borne n'est imposée.
- **Rate-limit global.** La limite (130 req/min) est globale, pas par appareil : un client bruyant peut pénaliser les autres. Une limite par clé serait plus juste.
- **Clé admin unique et statique.** Une seule clé partagée protège la gestion. Des jetons par administrateur (ou JWT à durée de vie courte) amélioreraient la traçabilité et la révocation.
- **Données de démonstration synthétiques.** Le script de seed génère des mesures plausibles pour tester les vues sémantiques sans collecte réelle ; il complète, sans remplacer, les sessions de terrain.